Лабораторная работа. Комплексная работа с MS SQL Server

Тема

Комплексная работа с возможностями Microsoft SQL Server: запросы, CTE, оконные функции, представления, хранимые процедуры, пользовательские функции и триггеры.

Цель работы

Закрепить основные навыки работы с Microsoft SQL Server и проверить умение применять изученные механизмы базы данных в рамках одной небольшой информационной системы.

В ходе работы необходимо:

- выполнять аналитические запросы;
- использовать CTE;
- применять оконные функции;
- создавать представления;
- создавать и вызывать хранимые процедуры;
- создавать пользовательские функции;
- создавать триггеры;
- проверять работу разработанных объектов.

Ситуация

Вы разрабатываете небольшую информационную систему колледжа.

В базе данных уже существуют:

- группы;
- студенты;
- оценки.

Необходимо добавить несколько элементов серверной логики, которые позволят получать аналитическую информацию, выполнять операции со студентами и автоматически контролировать изменения данных.

В работе используется база:

```
CollegeDB
```

и схема:

```
college
```

2. Подготовка базы данных

Используйте готовую базу `CollegeDB`.

Если база была удалена, сначала восстановите её из предоставленного преподавателем скрипта.

Проверьте наличие таблиц:

```sql
SELECT *
FROM college.groups;

SELECT *
FROM college.students;

SELECT *
FROM college.grades;
```

Основные поля таблиц:

`college.groups`

```
id
name
specialty
```

`college.students`

```
id
name
age
average_grade
group_id
```

`college.grades`

```
id
student_id
subject
grade
grade_date
```

Задание 1. Аналитический запрос

Необходимо получить список студентов с информацией:

```
Имя студента
Группа
Средний балл
Количество оценок
Средняя оценка по таблице grades
```

Используйте `JOIN`, `GROUP BY` и агрегатные функции.

В запросе не изменяйте данные базы.

Задание 2. CTE и рейтинг студентов

Используя CTE, сформируйте статистику студентов.

Для каждого студента необходимо рассчитать:

```
id
name
group_id
average_grade
```

После этого с помощью оконной функции:

```
RANK()
```

сформируйте общий рейтинг студентов по среднему баллу.

Результат должен содержать:

```
name
average_grade
rating
```

Студент с самым высоким средним баллом должен получить первое место.

Задание 3. Рейтинг внутри группы

Модифицируйте предыдущий запрос.

Теперь рейтинг должен рассчитываться отдельно для каждой группы.

Используйте:

```
PARTITION BY
```

В результате должны присутствовать:

```
student_name
group_name
```

```
average_grade
group_rating
```

Задание 4. Представление

Создайте представление:

```
college.StudentRating
```

Представление должно возвращать:

```
id
name
group_name
average_grade
```

и рейтинг студента внутри своей группы.

Используйте оконную функцию.

После создания проверьте представление:

```
SELECT *
FROM college.StudentRating;
```

Также выполните запрос:

```
SELECT *
FROM college.StudentRating
WHERE group_name = N'ИС-31';
```

Задание 5. Хранимая процедура

Создайте процедуру:

```
college.GetStudentsByMinGrade
```

Процедура должна принимать параметр:

```
@min_grade
```

и возвращать студентов, средний балл которых не меньше указанного значения.

Пример вызова:

```
EXEC college.GetStudentsByMinGrade
    @min_grade = 4.5;
```

Результат должен быть отсортирован по среднему баллу от большего к меньшему.

Проверьте процедуру минимум с двумя разными значениями параметра.

Задание 6. Пользовательская функция

Создайте scalar-функцию:

```
college.GetStudentStatus
```

Функция принимает:

```
@average_grade
```

и возвращает статус студента:

```
4.5 и выше  = Отличник
3.5–4.49    = Хорошист
ниже 3.5    = Требуется дополнительная подготовка
```

Пример:

```sql
SELECT
    name,
    average_grade,
    college.GetStudentStatus(average_grade) AS status
FROM college.students;
```

Функция должна возвращать именно одно значение для каждого переданного среднего балла.

Задание 7. Табличная функция

Создайте Inline Table-Valued Function:

```
college.GetStudentsByGroup
```

Функция должна принимать:

```
@group_id
```

и возвращать студентов указанной группы.

Результат должен содержать:

```
id
name
age
average_grade
```

Пример:

```sql
SELECT *
FROM college.GetStudentsByGroup(1);
```

Дополнительно выполните запрос:

```sql
SELECT *
FROM college.GetStudentsByGroup(1)
WHERE average_grade >= 4.5;
```

Задание 8. Триггер аудита

Создайте таблицу:

```
college.student_changes
```

со следующими полями:

```
id
student_id
old_average_grade
new_average_grade
change_date
```

`id` должен автоматически увеличиваться.

Затем создайте триггер:

```
college.trg_Students_AverageGradeAudit
```

Триггер должен срабатывать при:

```
UPDATE
```

таблицы:

```
college.students
```

Если изменяется `average_grade`, триггер должен записывать в `college.student_changes`:

- ID студента;

- старый средний балл;
- новый средний балл;
- дату изменения.

Для получения старого и нового значения необходимо использовать:

```
deleted
inserted
```

Проверка триггера

После создания триггера выполните:

```sql
UPDATE college.students
SET average_grade = average_grade + 0.1
WHERE id = 1;
```

Проверьте журнал:

```sql
SELECT *
FROM college.student_changes;
```

В нём должна появиться запись с:

```
student_id
старым значением
новым значением
датой изменения
```

После этого обязательно проверьте работу триггера сразу для нескольких студентов:

```sql
UPDATE college.students
SET average_grade = average_grade + 0.1
```

```
WHERE group_id = 1;
```

Снова выполните:

```
SELECT *
FROM college.student_changes;
```

Триггер должен корректно обработать все изменённые строки.

Итоговое задание

После выполнения всех предыдущих заданий сформируйте единый аналитический запрос.

Он должен показать:

```
Студент
Группа
Средний балл
Статус
Рейтинг в группе
Количество оценок
```

Используйте созданные ранее объекты базы данных там, где это возможно:

- представление;
- пользовательскую функцию;
- оконную функцию.

Требования к итоговому SQL-скрипту

Все выполненные задания должны быть сохранены в одном файле:

```
lastname_firstname_sqlserver.sql
```

Например:

```
Ivanov_Alexey_sqlserver.sql
```

Скрипт должен содержать:

1. аналитический запрос;
2. CTE и общий рейтинг;
3. рейтинг внутри группы;
4. создание `college.StudentRating`;
5. создание процедуры `college.GetStudentsByMinGrade`;
6. создание scalar-функции `college.GetStudentStatus`;
7. создание табличной функции `college.GetStudentsByGroup`;
8. создание таблицы `college.student_changes`;
9. создание триггера `college.trg_Students_AverageGradeAudit`;
10. команды проверки созданных объектов.

Перед каждым заданием рекомендуется использовать комментарий:

```
-- Задание 1
-- Аналитический запрос
```

```
-- Задание 2
-- CTE и рейтинг
```

и так далее.

Результат работы

В результате выполнения лабораторной работы в базе данных должны появиться следующие объекты:

```
CollegeDB
|
```

```
└── college
    │
    ├── StudentRating
    │
    ├── GetStudentsByMinGrade
    │
    ├── GetStudentStatus
    │
    ├── GetStudentsByGroup
    │
    ├── student_changes
    │
    └── trg_Students_AverageGradeAudit
```

Кроме того, итоговый SQL-скрипт должен содержать аналитические запросы с CTE и оконными функциями.

Итог

В этой работе вы объединяете изученные возможности SQL Server в одной небольшой информационной системе.

Главная задача лабораторной — не просто написать отдельные SQL-команды, а показать, что вы понимаете назначение различных объектов базы данных и можете выбрать подходящий механизм для конкретной задачи.

Это последняя лабораторная работа перед промежуточной аттестацией.